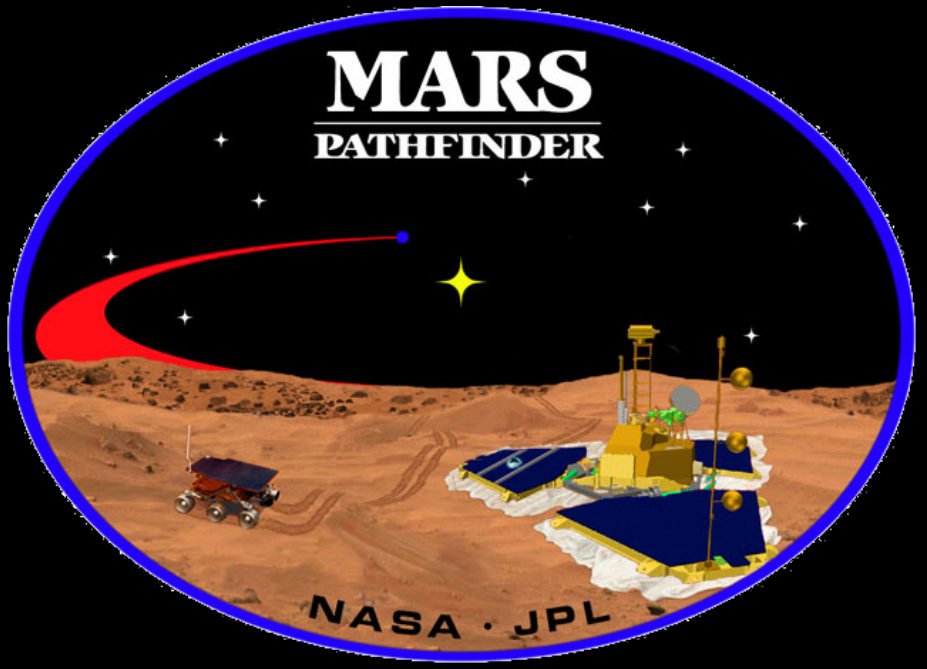
¿Por qué modelar y analizar programas concurrentes?

- Son notoriamente difíciles de construir correctamente.
- Los errores son difíciles de replicar.
- Pero también…
  - Establecer un modelo matemático que los caracteriza es interesante a nivel disciplina





Therac-25

Preguntas de Interés

(Para Programas Concurrentes)

Compartidas con programas secuenciales

- Post-condiciones
- Invariante de ciclo
- Aserciones en código
- Terminación
- …

Particulares de programas concurrentes

- Deadlock
- Livelock
- Interferencia
- Fairness
- Atomicidad
- Propiedades de dominio específico
- ….

ANTONY HOARE

Made enduring contributions to programming language design and definition

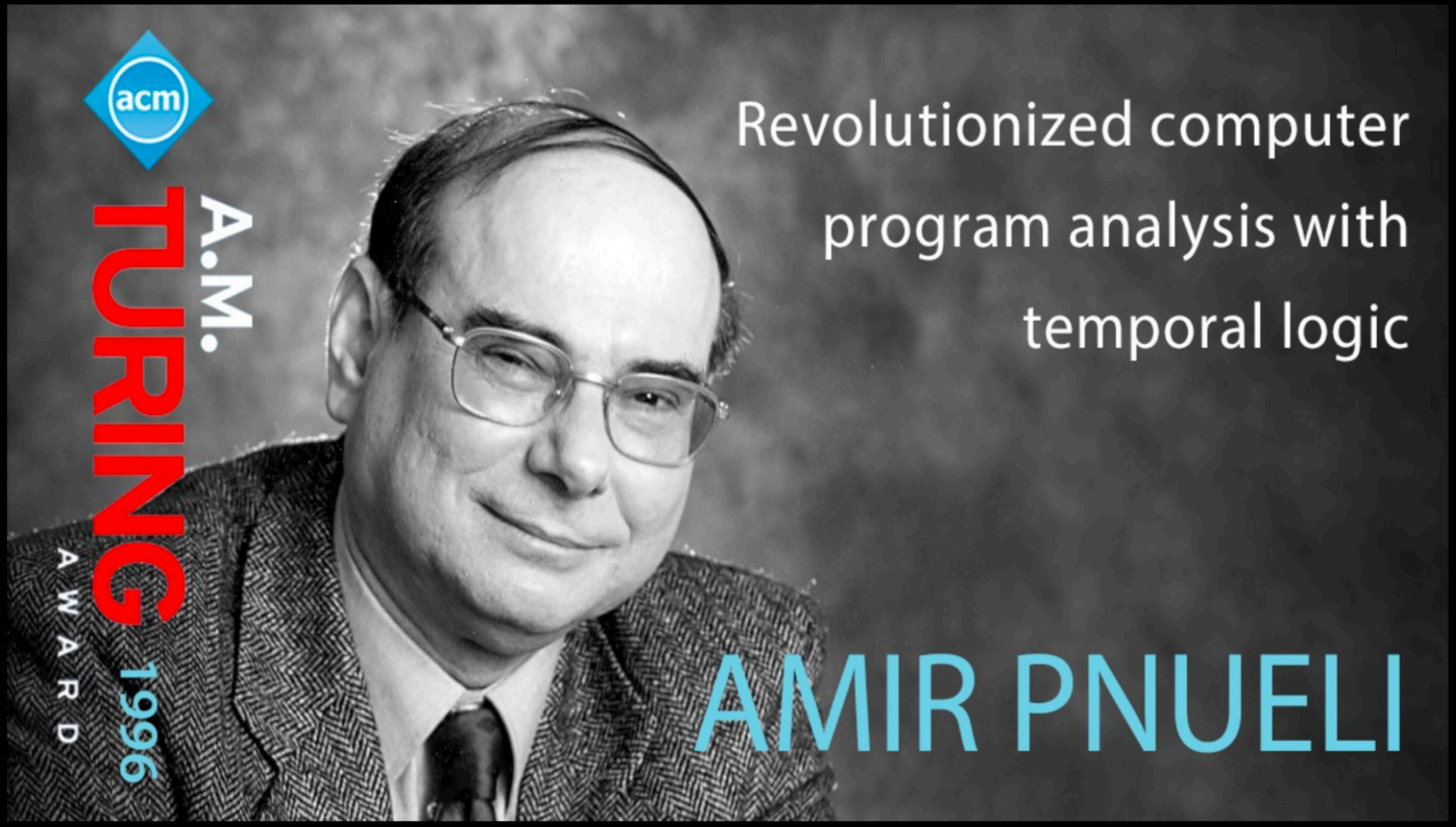


A.M. TURING AWARD 1980

A.M. TURING AWARD 2013

Reimagined distributed and concurrent systems.

LESLIE LAMPORT


acm
A.M.
TURING
AWARD
1996
Revolutionized computer
program analysis with
temporal logic
AMIR PNUELI



https://dl.acm.org/doi/pdf/10.1145/1592761.1592781

Modelos de Concurrencia

- Cual sería el WHILE de programas concurrentes?
- Algebras de Proceso.
  - Communicating Sequential Processes (CSP).
    - Tony Hoare.
    - Go language
  - Finite State Processes (FSP)
    - Una variante de CSP que garantiza modelos finitos.

Sintaxis, Semántica y Denotación



MTSA

- Originalmente LTSA - The Labelled Transition System Analyser. Imperial College.
- Compila FSP en sistemas de transición etiquetadas y provee múltiples formas de análisis.
- Evolución en dc.uba.ar para:
  - Modal Transition Systems
  - Markov Decision Processes
  - Síntesis de Controladores de Eventos Discretos
- http://mtsa.dc.uba.ar/

Sistemas de Transición Etiquetadas

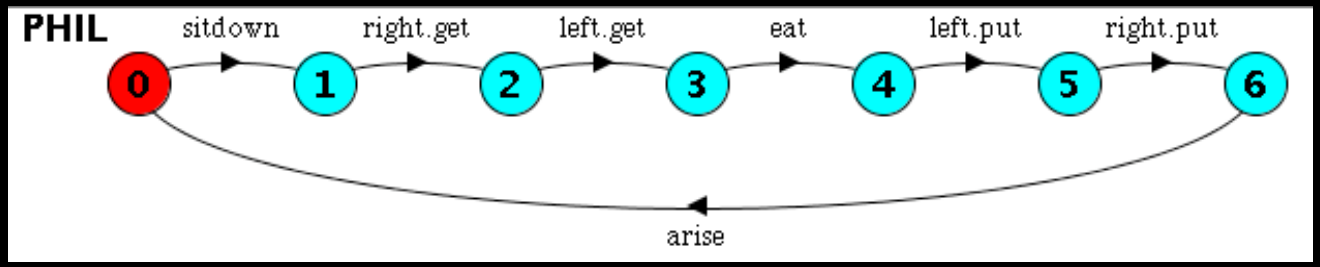
- Las etiquetas modelan interacciones de un proceso con su entorno
- Hay una acción especial “tau” que modela cómputo interno del proceso, que no es observable desde el entorno.
- Hay una cantidad finita de estados

Definición. *(LTS)* Sea $Estados$ el universo de estados, Act el universo de acciones observables, y $Act_\tau = Act \cup \{\tau\}$. Un LTS es una tupla $P = (S, A, \Delta, s_0)$, donde $S \subseteq Estados$ y $A \subseteq Act_\tau$ son conjuntos finitos, $\Delta \subseteq (S \times A \times S)$ es un conjunto de transiciones etiquetadas, y $s_0 \in S$ es el estado inicial. Definimos el alfabeto de comunicación de P como $\alpha P = A \setminus \{\tau\}$.

- Usaremos el término LTS (Labelled Transition System)

Un LTS

PHIL = <{0 1, 2, 3, 4, 5, 6},
{sitdown, right.get, left.get, eat, left.put, right.put, arise},
{(0, sitdown, 1), (1, right.get, 2), (2, left.get, 3), (3, eat, 4), (5, left.put, 6), (6, arise, 0)},
0}



Definición. *(Ejecuciones y Trazas)* Una ejecución de un LTS $P = (S, A, \Delta, s_0)$ es una secuencia $s_0, \ell_0, s_1, \ell_1, \ldots$, donde para cada $i \geq 0$ tenemos $(s_i, \ell_i, s_{i+1}) \in \Delta$. Una secuencia $l_0, l_1, l_2, \ldots$ es una traza de P si es proyección sobre αP de una ejecución de P.

0, sitdown, 1, right.get, 2, left.get, 3, eat, 4, left.put, 5, right.put, 6, arise, 0, sitdown, …

sitdown, right.get, left.get, eat, left.put, right.put, arise, sitdown, …

Modelado Sistemas Concurrentes

Parte 3/5

Lenguaje FSP



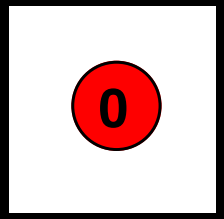
Quick reference:
https://www.doc.ic.ac.uk/~jnm/book/ltsa/Appendix-A-2e.html

Sintaxis y semántica
https://www.doc.ic.ac.uk/~jnm/LTSdocumention/FSP-notation.html

FSP - El proceso Deadlock

STOP es un proceso (y palabra reservada) que describe el proceso que es incapaz de interactuar con su entorno.

```
NadaDeNada = STOP.
```

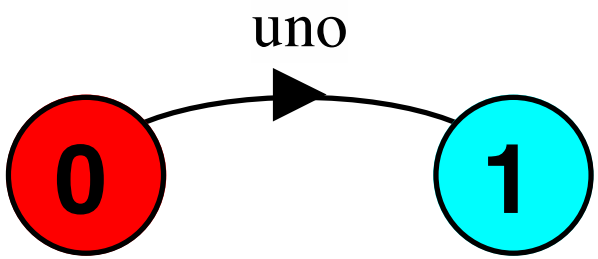


$lts(\mathbf{STOP}) = <\{s\}, \{\}, \{\}, s>$

FSP - action prefix

Si **x** es una acción y **P** un proceso entonces **(x-> P)** describe un proceso que inicialmente interactúa a través de la acción **x** y luego se comporta como **P**.

```
UnicaVez = (uno -> STOP).
```



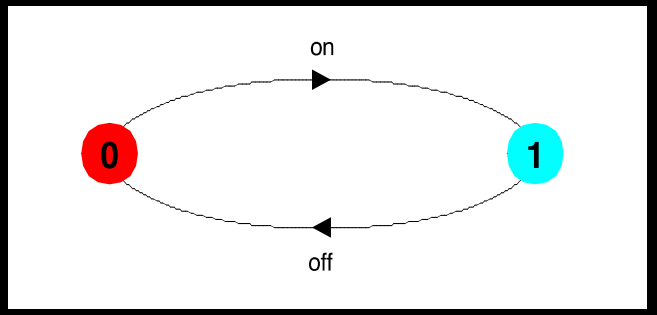
$lts(\mathbf{UnicaVez}) = <\{s, p\}, \{uno\}, \{(s, uno, p)\}, s>$

```
Dosveces = (uno -> dos -> STOP).
```

FSP - Recursión

Repetición de comportamiento se modela con recursión:

```
SWITCH = (on->off->SWITCH).
```



$lts(\mathbf{Switch}) = \langle \{s, p\}, \{on, off\}, \{(s, on, p), (p, off, s)\}, s \rangle$

Sub-Procesos

a.k.a. Definiciones locales a un Proceso

```
SWITCH = OFF,
OFF    = (on -> ON),
ON     = (off-> OFF).

SWITCH = OFF,
OFF    = (on ->(off->OFF)).

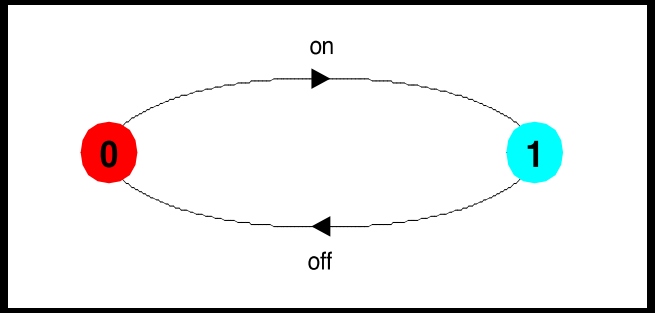
SWITCH = (on->off->SWITCH).
```

La coma introduce un subproceso.

El punto termina la definición de un proceso

Acciones/eventos comienzan con minúscula.

Procesos comienzan con mavúscula

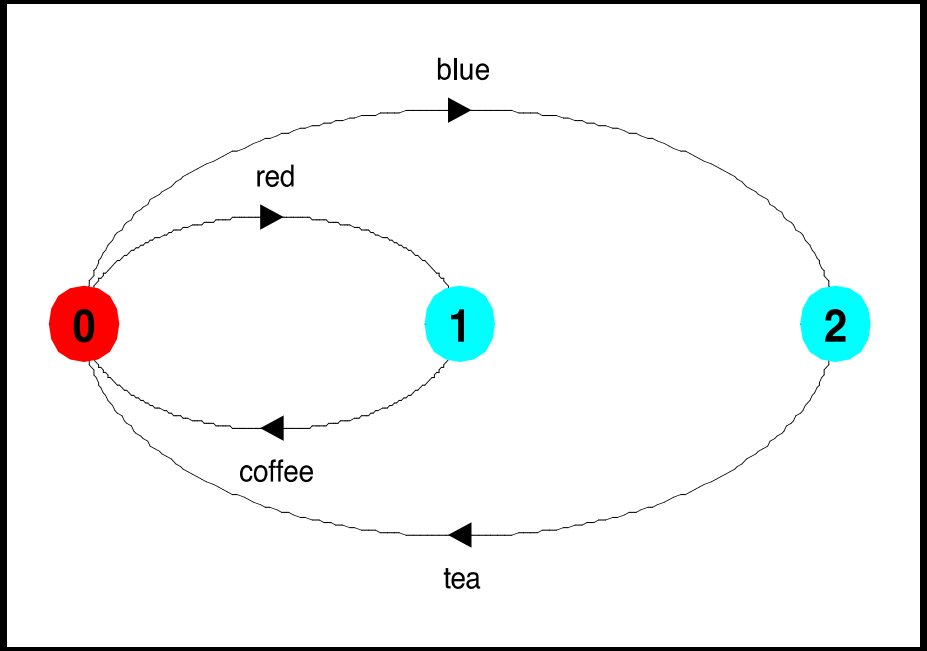


FSP - Alternativas

Si **x** e **y** son acciones entonces

(x-> P | y-> Q)

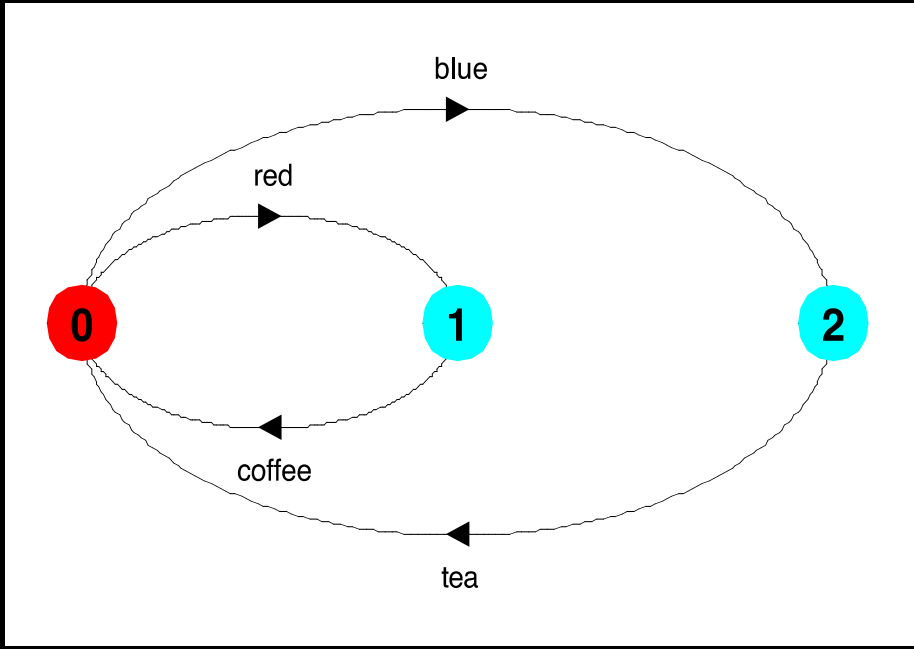
describe un proceso que inicialmente es capaz de interactuar a través de las acciones **x** o **y**.
El proceso pasa a comportarse como **P** o **Q** según ocurra **x** o **y.**




```
DRINKS = (red->coffee->DRINKS
         |blue->tea->DRINKS
         ).
```

FSP - Alternativas

```
DRINKS = (red->coffee->DRINKS
         |blue->tea->DRINKS
         ).
```



Ejecuciones y Trazas?

Quién elige que alternativa tomar?

No hay una distinción entre output e input?

Qué significa que “red” no esté habilitado en 1?

FSP - Procesos y acciones indexadas

```
BUFF = (in[i:0..3]->out[i]-> BUFF).
```

equivalente a

```
BUFF = (in[0]->out[0]->BUFF
       |in[1]->out[1]->BUFF
       |in[2]->out[2]->BUFF
       |in[3]->out[3]->BUFF
       ).
```

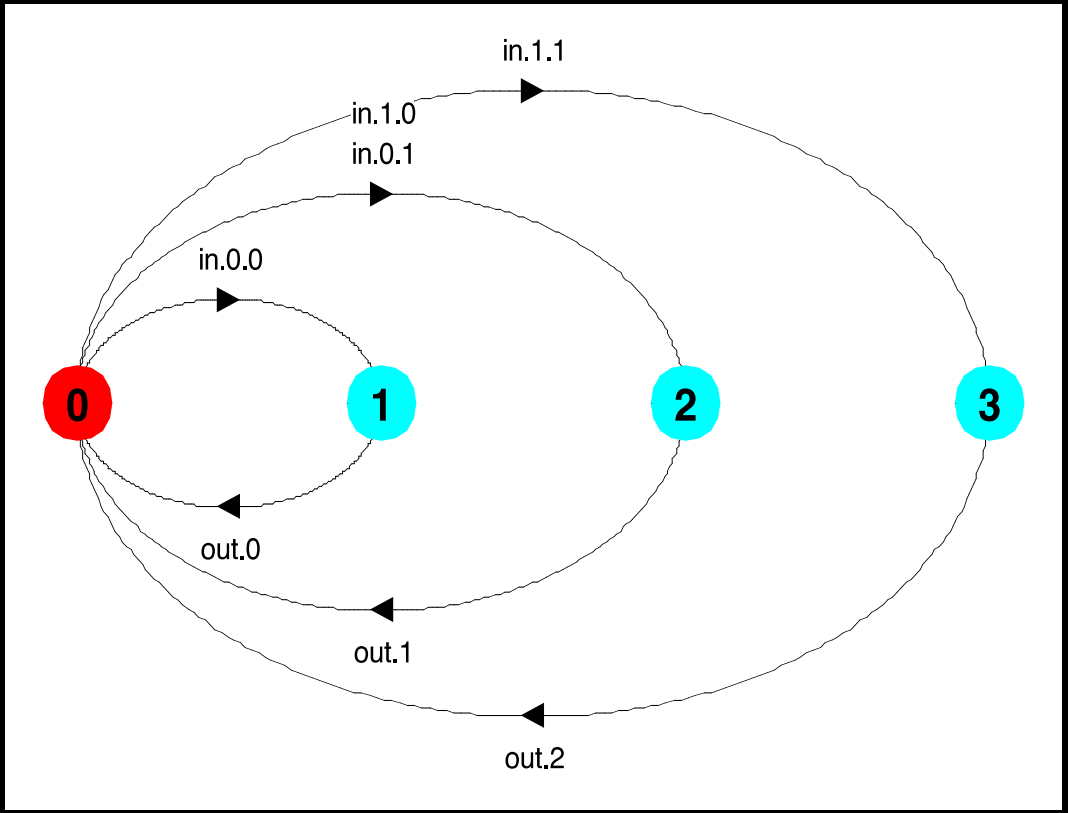
o usando un parámetro de proceso con un valor por defecto:

```
BUFF(N=3) = (in[i:0..N]->out[i]-> BUFF).
```

Azúcar sintáctico

FSP - Declaración de Constantes y Rangos

Indices pueden usarse para modelar cómputo:




```
const N = 1
range T = 0..N
range R = 0..2*N

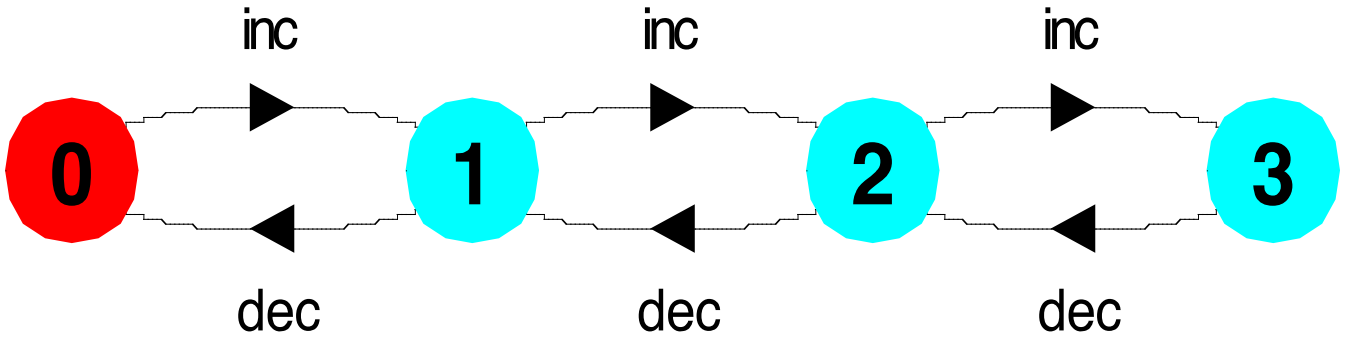
SUM          = (in[a:T][b:T]->TOTAL[a+b]),
TOTAL[s:R] = (out[s]->SUM).
```

Azúcar sintáctico

FSP - Guardas

La alternativa **(when B x -> P | y -> Q)** significa que cuando la guarda **B** es verdadera entonces tanto **x** como **y** pueden ser elegidas, caso contrario, si **B** es falso entonces la acción **x** no puede ser elegida.

```
COUNT  (N=3)    = COUNT[0],
COUNT[i:0..N] = (when(i<N) inc->COUNT[i+1]
                |when(i>0) dec->COUNT[i-1]
                ).
```

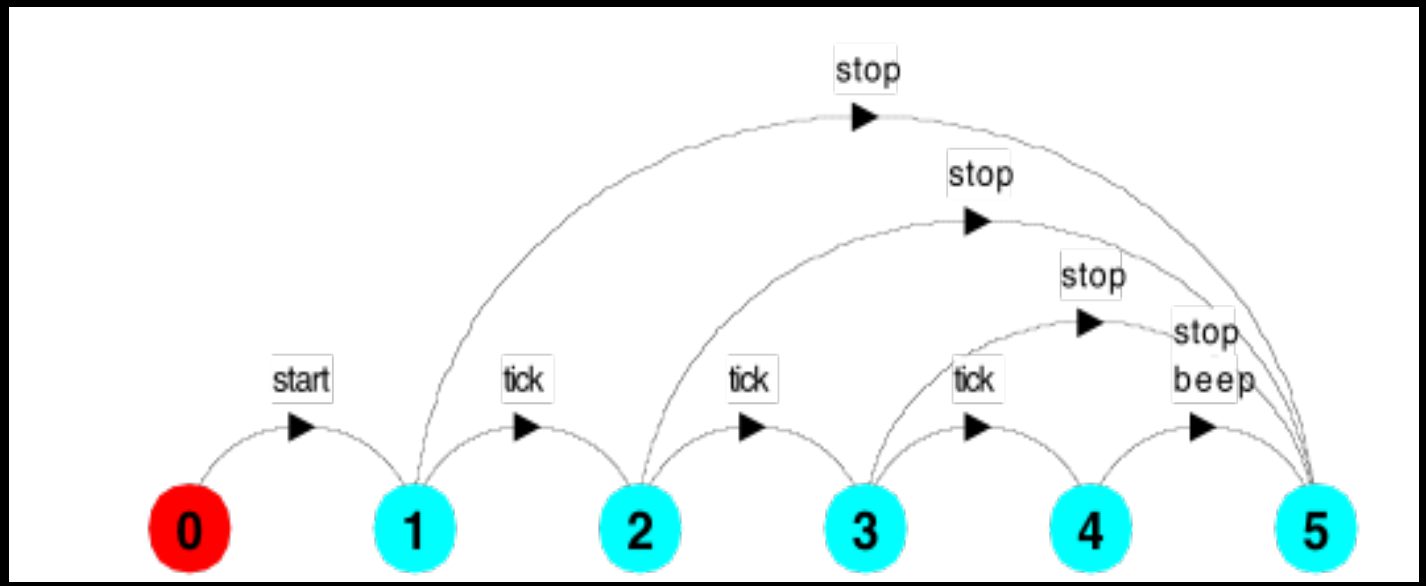


Azúcar sintáctico

FSP - Guardas

Una alarma que cuenta N ticks y hace beep y que puede ser parado en cualquier momento

```
COUNTDOWN (N=3) = (start->COUNTDOWN[N]),
COUNTDOWN[i:0..N] =
            (when(i>0) tick->COUNTDOWN[i-1]
            |when(i==0)beep->STOP
            |stop->STOP
            ).
```



Azúcar sintáctico

¿Vínculo con teoría de lenguajes y autómatas?

Composición en paralelo

Si P y Q son procesos entonces (P||Q) representa la ejecución concurrente de P and Q.

```
ITCH  = (scratch->STOP).
CONVERSE = (think->talk->STOP).

||CONVERSE_ITCH = (ITCH || CONVERSE).
```

$$lts(\mathtt{P}\,||\,\mathtt{Q}) = lts(\mathtt{P}) \parallel lts(\mathtt{Q})$$

La composición paralela es una forma de construir un modelo que tiene todas las ejecuciones posibles de la ejecucíon paralela de dos procesos. Una vez compuesto, es un proceso sequencial.

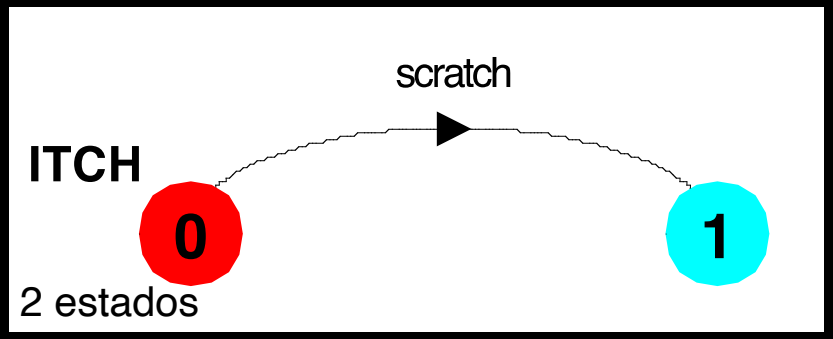
Composición en paralelo de LTS

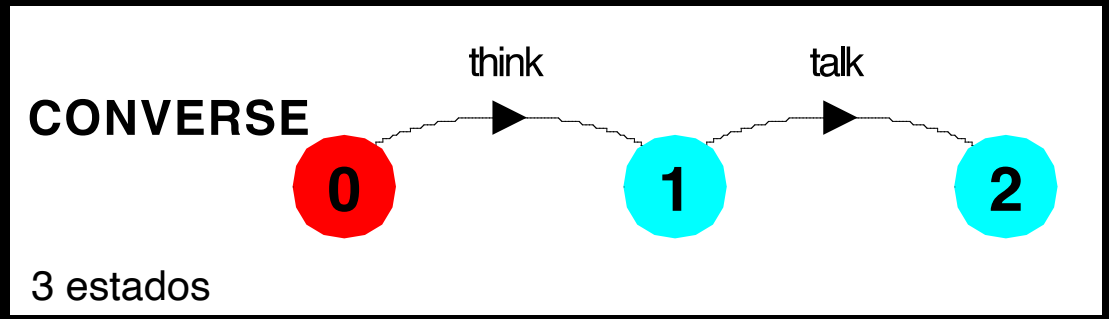
Definición. *(Composición Paralela)* Sean los LTS $M = (S_M, A_M, \Delta_M, s_0^M)$ y $N = (S_N, A_N, \Delta_N, s_0^N)$. La *composición paralela* $\|$ es un operador simétrico tal que $M \| N$ es el LTS $(S_M \times S_N, A_M \cup A_N, \Delta, (s_0^M, s_0^N))$, donde Δ es la relación más chica que satisface las siguientes reglas (con $\ell \in A_M \cup A_N$):

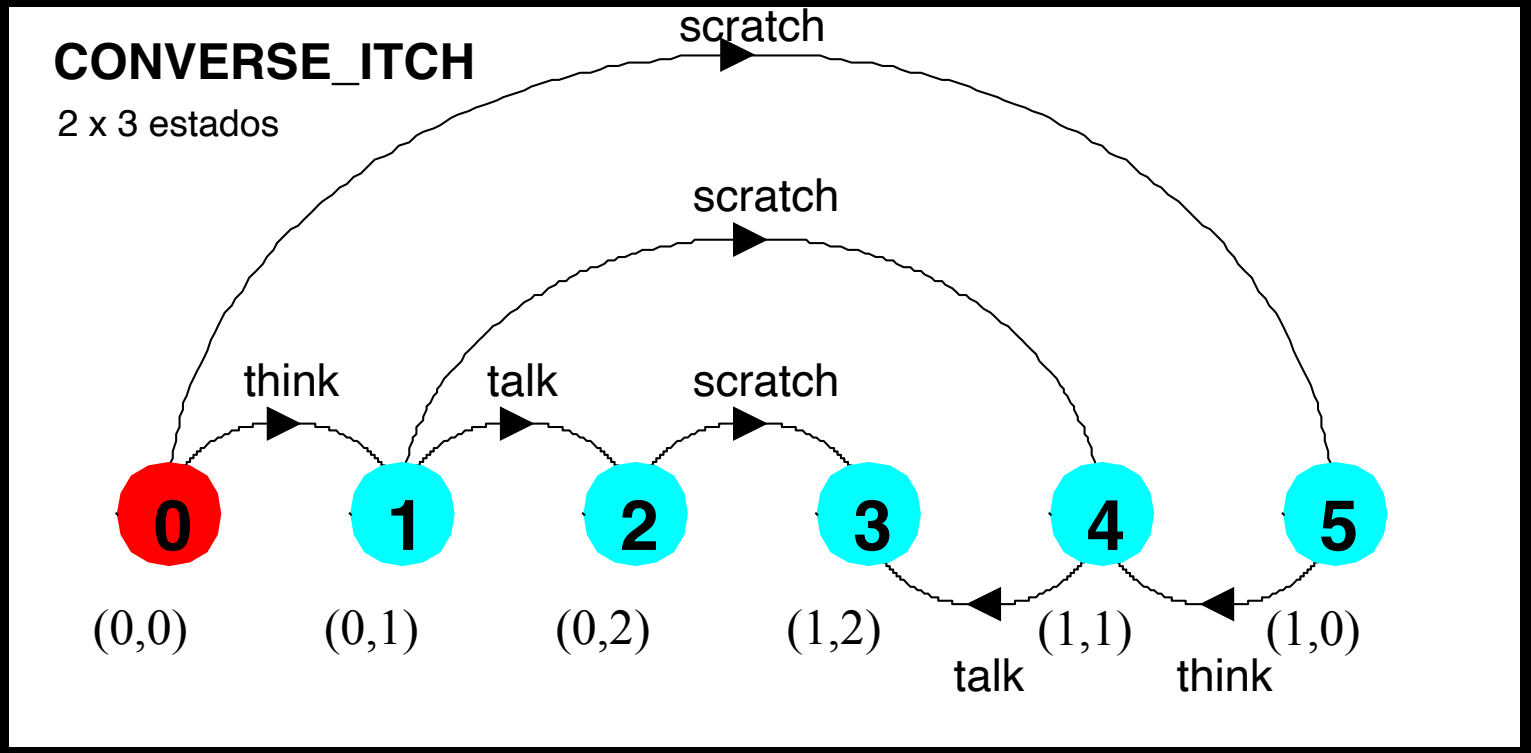
$$\frac{(s, \ell, s') \in \Delta_M}{((s,t), \ell, (s',t)) \in \Delta} \; \ell \notin \alpha N \qquad \frac{(t, \ell, t') \in \Delta_N}{((s,t), \ell, (s,t')) \in \Delta} \; \ell \notin \alpha M$$

$$\frac{(s, \ell, s') \in \Delta_M, \; (t, \ell, t') \in \Delta_N}{((s,t), \ell, (s',t')) \in \Delta} \; \ell \in \alpha M \cap \alpha N$$

Interleaving



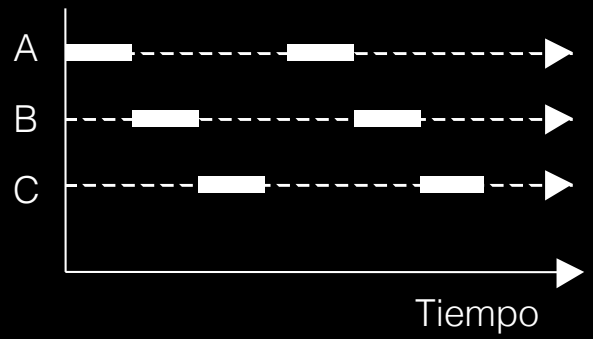




Paralelismo vs Concurrencia

Concurrencia:
Procesamiento lógicamente simultáneo.
No implica múltiples unidades de procesamiento.
Requiere ejecución “interleaved” en un UP.

Paralelismo
Procesamiento físicamente simultáneo.
Involucra múltiples UPs.



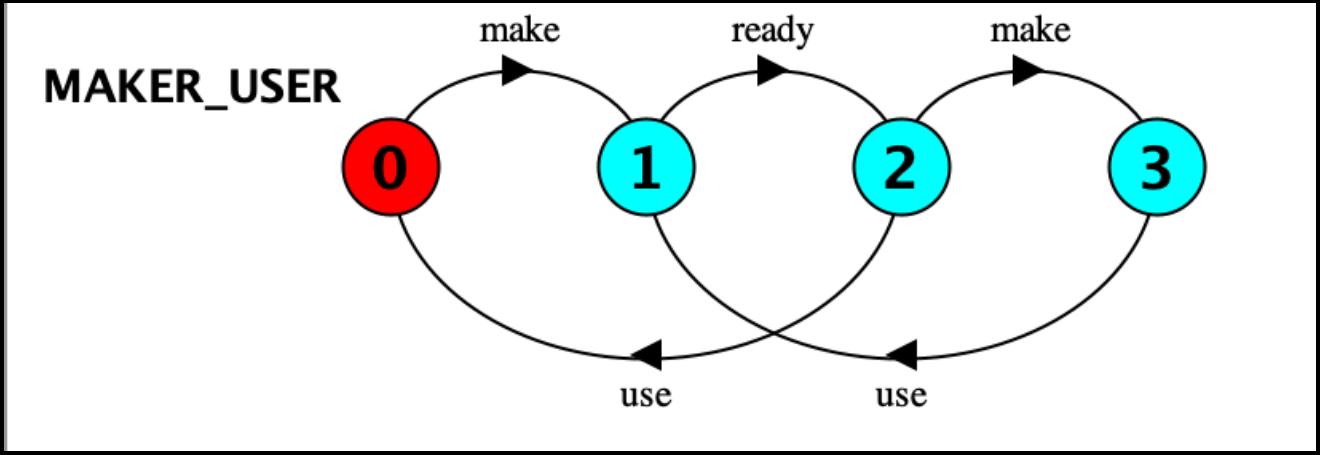
El modelo de concurrencia de FSP es uno de “interleaving”. No puede modelar la ocurrencia de dos eventos en simultáneo.

Modelos para “true concurrency” ver redes de petri o estructuras de kripke.

Acciones compartidas

```
MAKER = (make->ready->MAKER).
USER  = (ready->use->USER).

||MAKER_USER = (MAKER || USER).
```

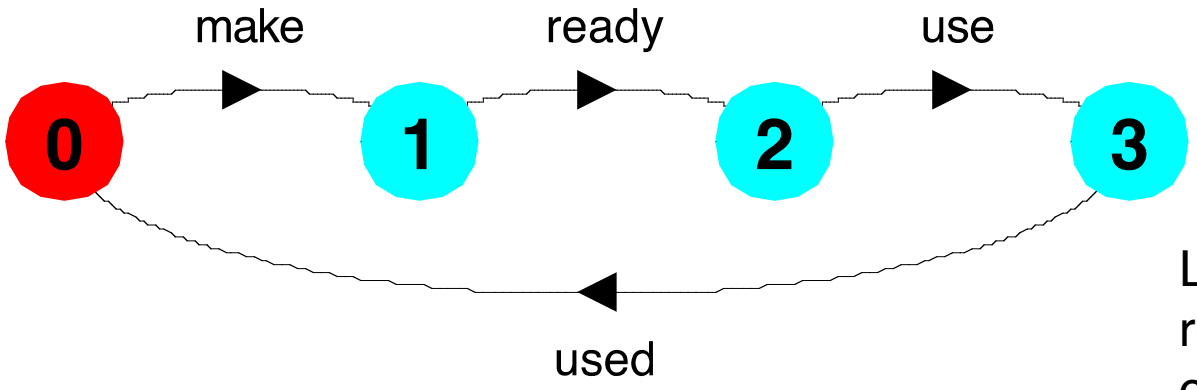


Modelado de Handshake

```
MAKERv2 = (make->ready->used->MAKERv2).
USERv2  = (ready->use->used ->USERv2).

||MAKER_USERv2 = (MAKERv2 || USERv2).
```

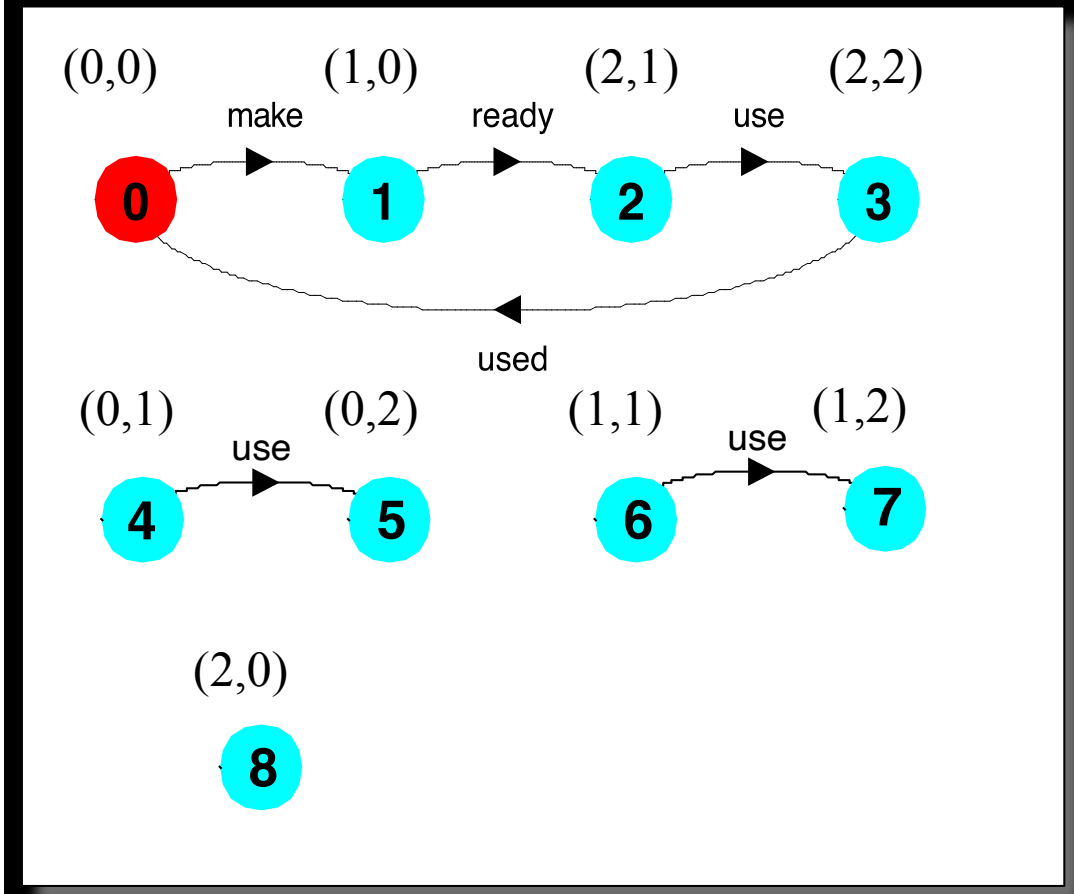
3 x 3 = 9 estados?

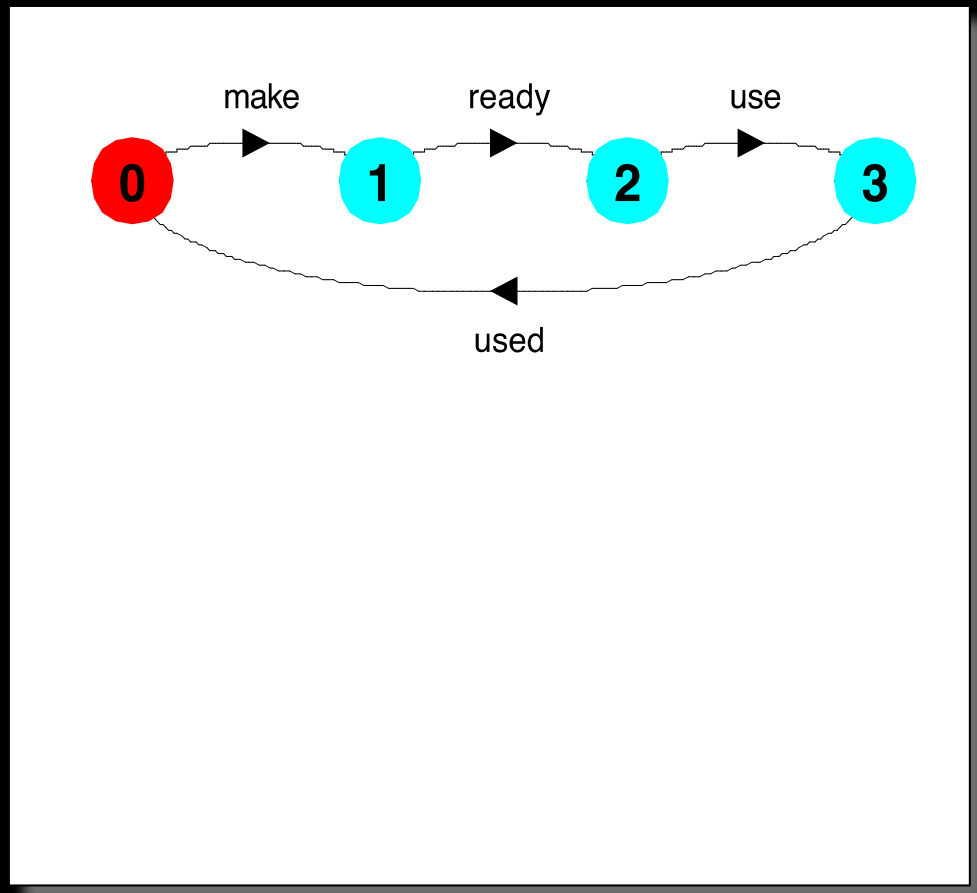


La interacción restringe el comportamiento global.

Equivalencia de LTS

(MAKERv2 || USERv2)



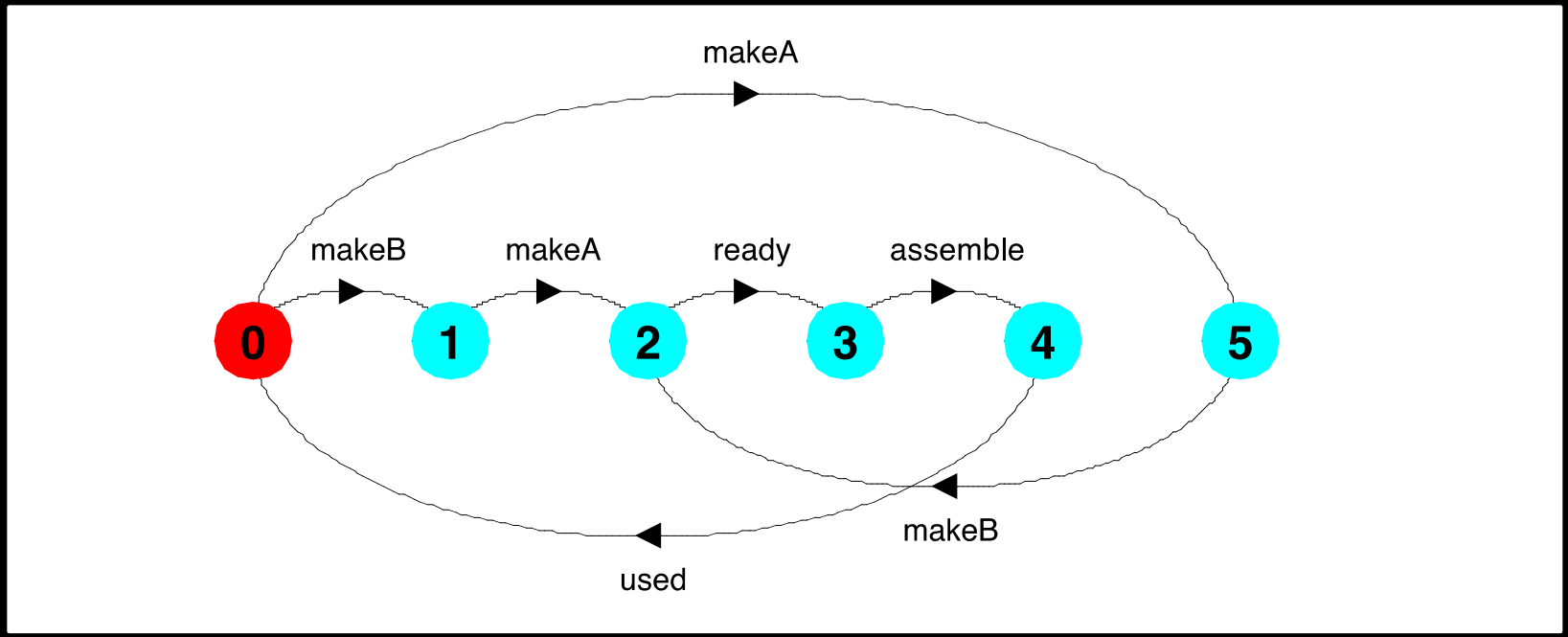


Son equivalentes estos dos LTS?

Sincronización Múltiple

```
MAKE_A    = (makeA->ready->used->MAKE_A).
MAKE_B    = (makeB->ready->used->MAKE_B).
ASSEMBLE  = (ready->assemble->used->ASSEMBLE).

||FACTORY = (MAKE_A || MAKE_B || ASSEMBLE).
```



Procesos compuestos

Un proceso compuesto es la composición en paralelo de procesos. Un proceso compuesto puede usarse en nuevas composiciones.

```
||MAKERS = (MAKE_A || MAKE_B).

||FACTORY = (MAKERS || ASSEMBLE).
```

Por sustitución y aplicando conmutatividad y asociatividad obtenemos:

```
||FACTORY = (MAKE_A || MAKE_B ||
ASSEMBLE).
```

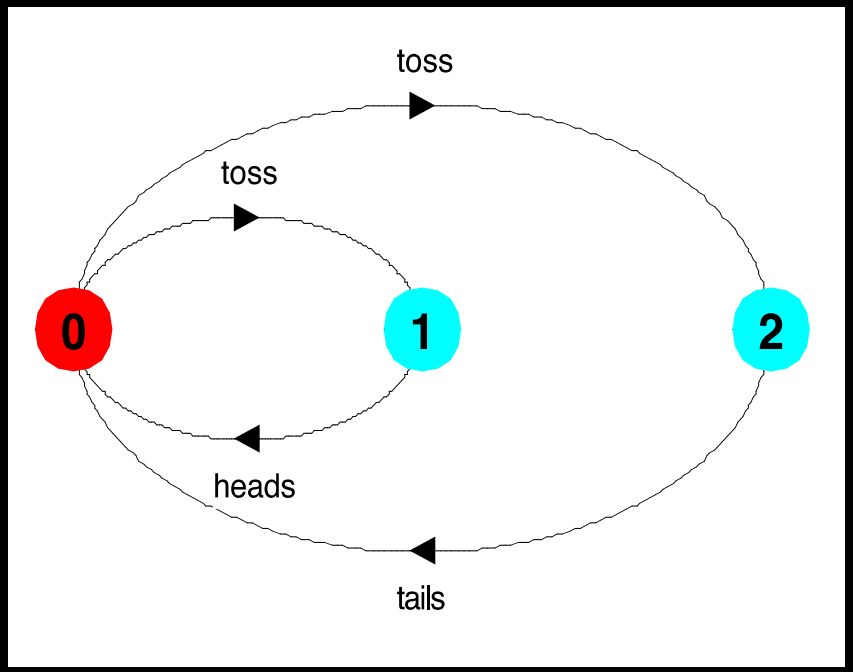
Modelado Sistemas Concurrentes

Parte 4/5

Elección no-determinística

El proceso (x-> P | x -> Q) describe una elección no determinística entre P or Q.

```
COIN = (toss->HEADS | toss->TAILS),
HEADS = (heads->COIN),
TAILS = (tails->COIN).
```



Ojo, No determinismo no es lo mismo que equi-probable

No determinismo

```
COIN = (toss->HEADS|toss->TAILS),
HEADS= (heads->COIN),
TAILS= (tails->COIN).


PLAY = (toss-> (heads -> HEADS1 | tails -> TAILS1)),
TAILS1 = (toss -> (heads -> HEADS1 | tails -> TAILSWIN)),
TAILSWIN = (twoTails -> end -> STOP),
HEADS1  = (toss -> (heads -> HEADSWIN | tails -> TAILS1)),
HEADSWIN = (twoHeads -> end -> STOP).

||JUEGO = (PLAY || COIN).
```

Juego con moneda (v2)

```
COIN = (toss->HEADS|toss->TAILS),
HEADS= (heads->COIN),
TAILS= (tails->COIN).


REPETICIONES(MAX = 3) = (toss-> (heads -> HEADS[1] | tails -> TAILS[1])),
TAILS[i:1..MAX-1] = (toss -> (heads -> HEADS[1] | tails -> TAILS[i+1])),
TAILS[MAX] = (fin -> STOP),
HEADS[i:1..MAX-1]  = (toss -> (heads -> HEADS[i+1] | tails -> TAILS[1])),
HEADS[MAX] = (fin -> STOP).

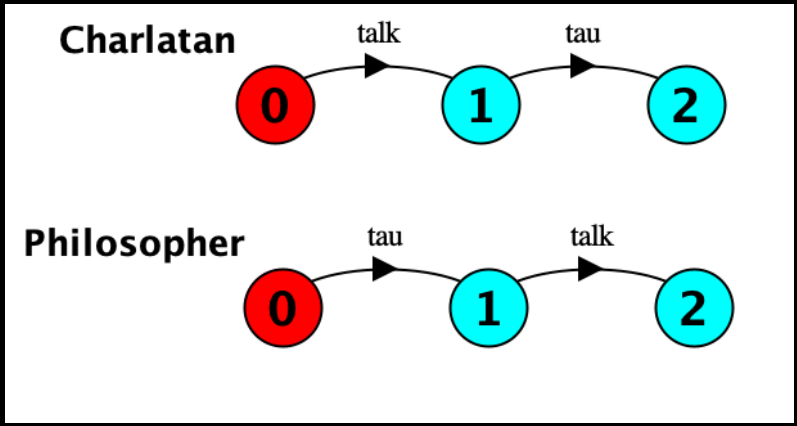
||JUEGO = (REPETICIONES(2) || COIN).
```

Ocultamiento de acciones

En FSP no se puede usar el literal “tau” para representar la acción que representa cómputo interno.

```
Charlatan = (talk -> think -> STOP)\{think}.

Philosopher = (think -> talk -> STOP)@{talk}.
```



Definición. *(Ocultamiento)* Sea el LTS $P = (S, A, \Delta, s_0)$ y conjunto de acciones $B \subseteq Act$. El ocultamiento de B en P (escrito como $P \setminus B$) es el LTS $(S, (A \setminus B) \cup \{\tau\}, \Delta', s_0)$ donde Δ' es el conjunto más chico que satisface:

$$\frac{(s, \ell, s') \in \Delta}{(s, \ell, s') \in \Delta'}\ \ell \notin B \qquad \frac{(s, \ell, s') \in \Delta}{(s, \tau, s') \in \Delta'}\ \ell \in B$$

Definición. *(Composición Paralela)* Sean los LTS $M = (S_M, A_M, \Delta_M, s_0^M)$ y $N = (S_N, A_N, \Delta_N, s_0^N)$. La *composición paralela* $\|$ es un operador simétrico tal que $M \| N$ es el LTS $(S_M \times S_N, A_M \cup A_N, \Delta, (s_0^M, s_0^N))$, donde Δ es la relación más chica que satisface las siguientes reglas (con $\ell \in A_M \cup A_N$):

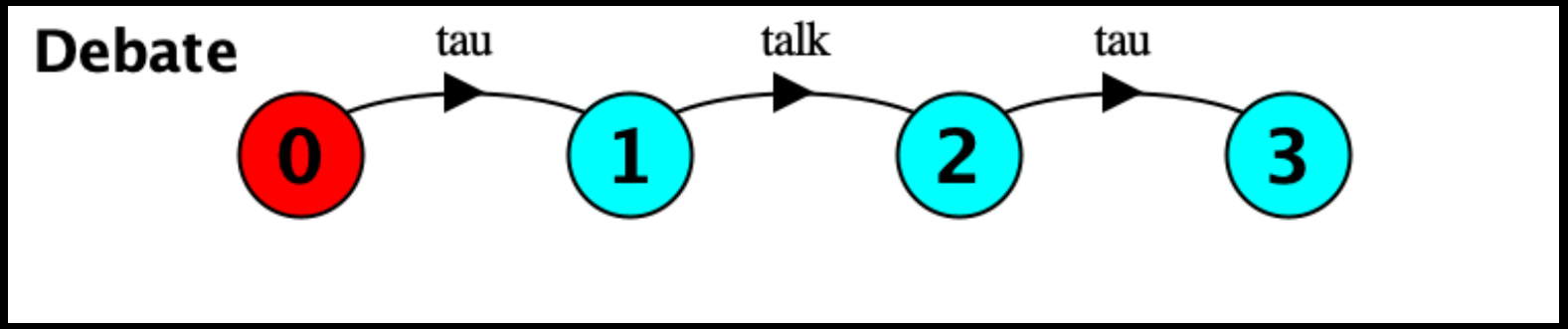
$$\frac{(s, \ell, s') \in \Delta_M}{((s,t), \ell, (s',t)) \in \Delta} \; \ell \notin \alpha N \qquad \frac{(t, \ell, t') \in \Delta_N}{((s,t), \ell, (s,t')) \in \Delta} \; \ell \notin \alpha M$$

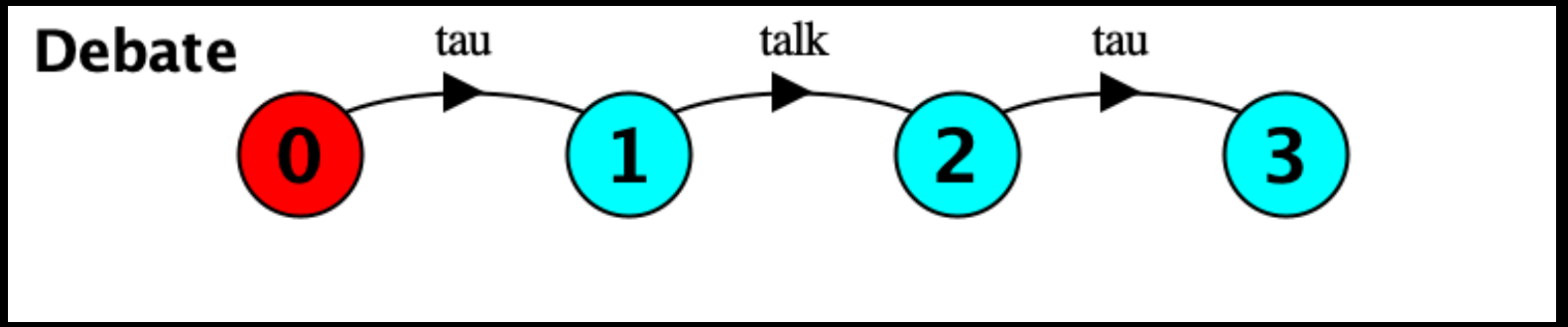
$$\frac{(s, \ell, s') \in \Delta_M, \;\; (t, \ell, t') \in \Delta_N}{((s,t), \ell, (s',t')) \in \Delta} \; \ell \in \alpha M \cap \alpha N$$

Composición Paralela

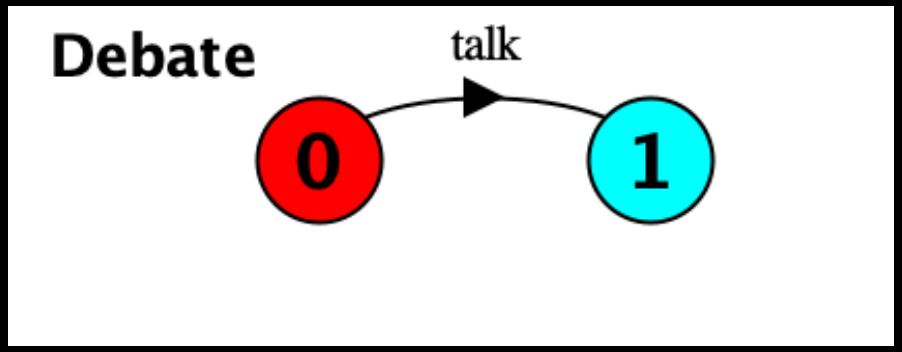
```
Charlatan = (talk -> think -> STOP)\{think}.
Philosopher = (think -> talk -> STOP)@{talk}.

||Debate = (Philosopher || Charlatan).
```





Minimización que preserva semántica

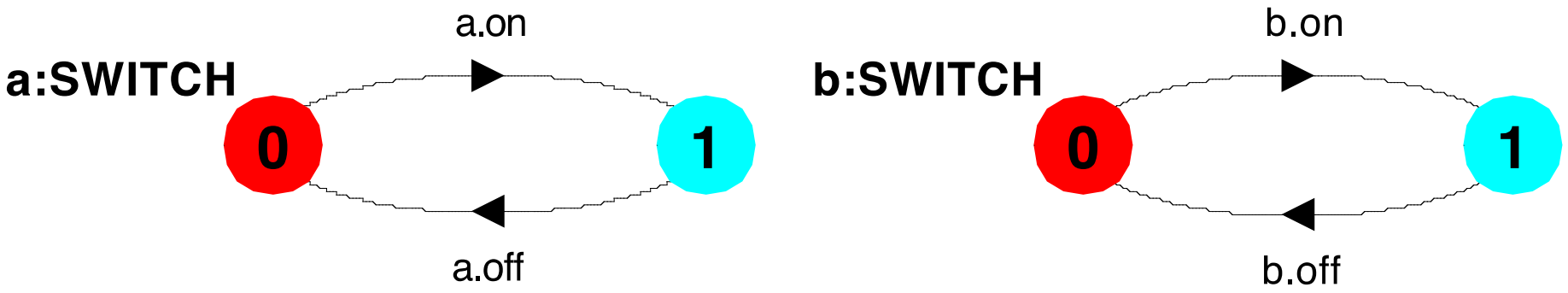


Etiquetado de Procesos

a:P prefija a cada acción en el alfabeto de P con a.

Dos **instancias** de un proceso switch:

```
SWITCH =  (on->off->SWITCH).
||TWO_SWITCH = (a:SWITCH || b:SWITCH).
```



Un vector de **instancias** del proceso switch:

```
||SWITCHES(N=3) = (forall[i:1..N] s[i]:SWITCH).
||SWITCHES(N=3) = (s[i:1..N]:SWITCH).
```

Prefijado de Procesos

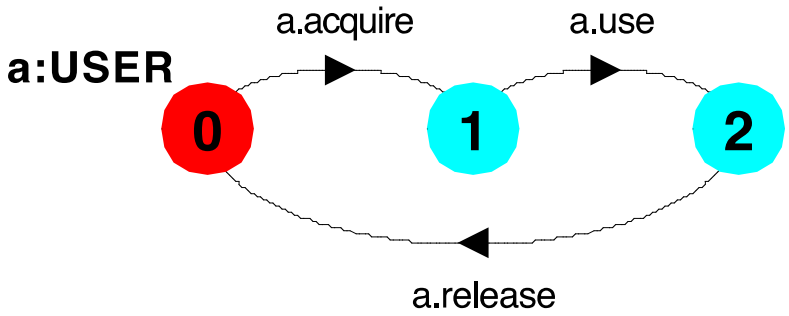
{a1,..,ax}::P remplaza cada acción n del alfabeto de P con etiquetas a1.n,…,ax.n. Además, cada transición (n->X) en P se remplaza con ({a1.n,…,ax.n} ->X).

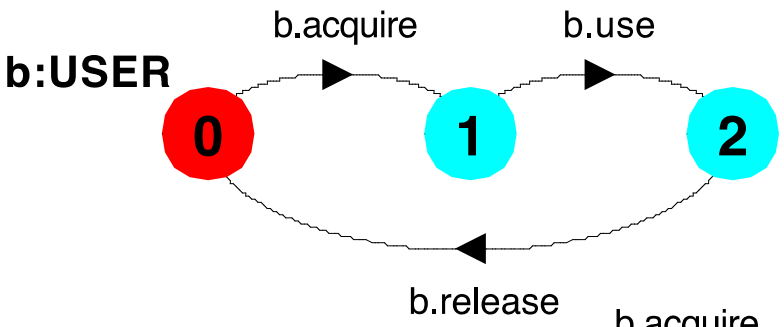
Prefijado de procesos es util para modelar recursos **compartidos** resources:

```
RESOURCE = (acquire->release->RESOURCE).
USER = (acquire->use->release->USER).

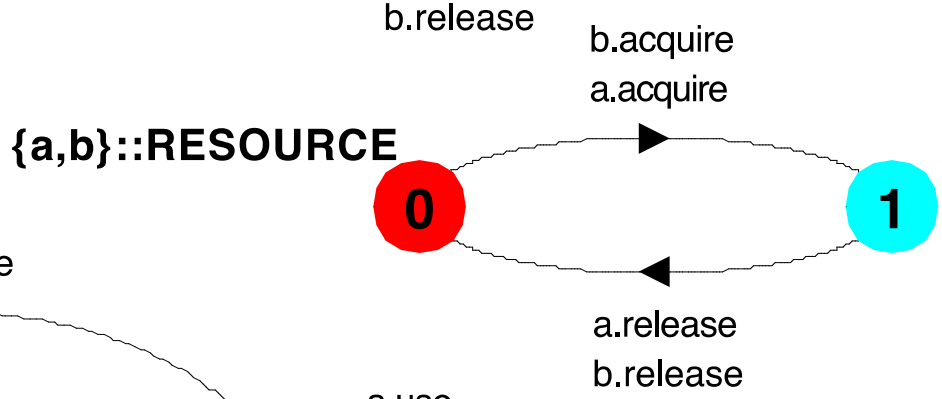
||RESOURCE_SHARE = (a:USER || b:USER
                        || {a,b}::RESOURCE).
```

Compartiendo recursos





Como asegura el modelo que el mismo usuario que adquiere un recurso es el que lo libera?



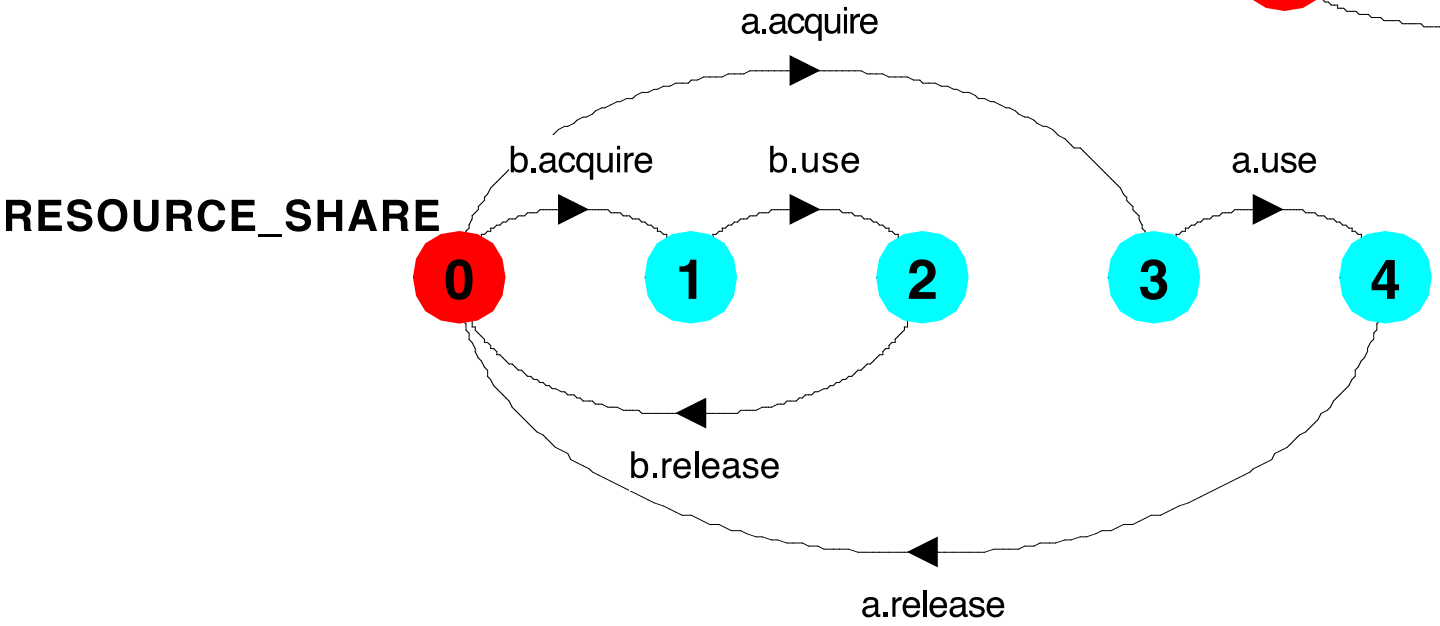
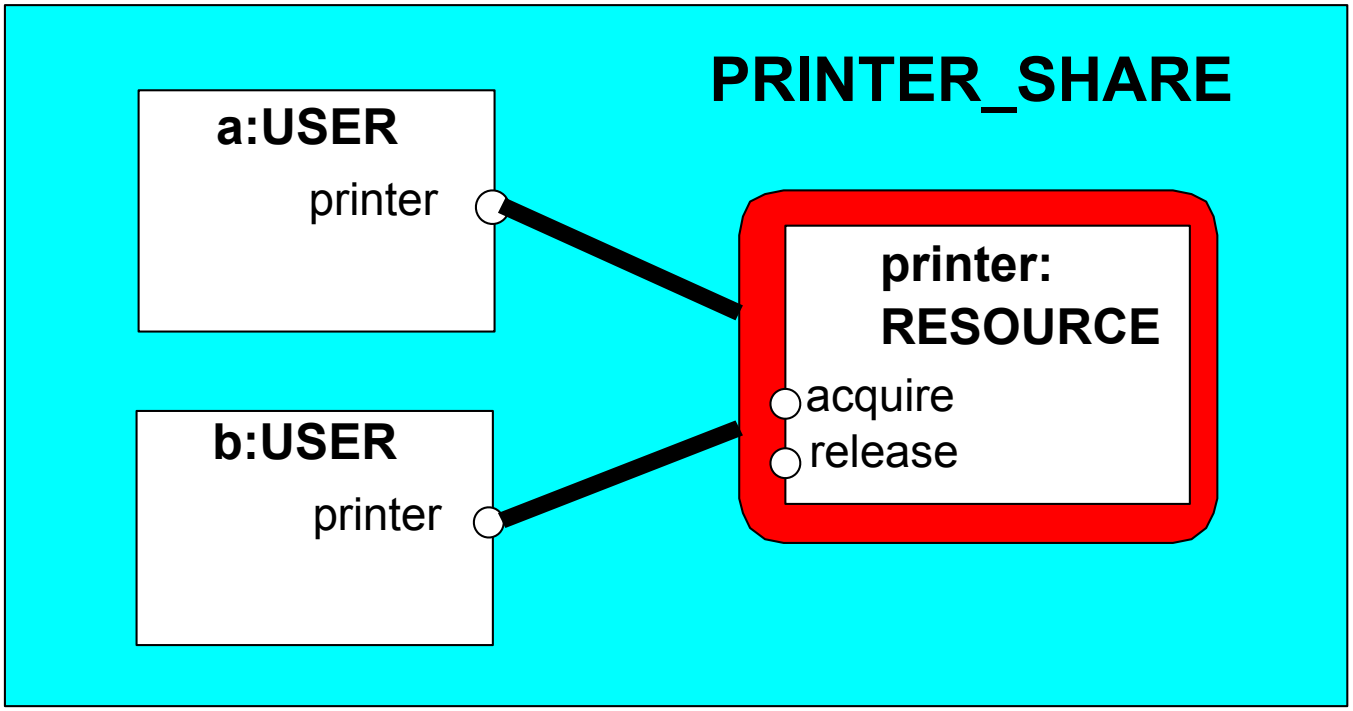


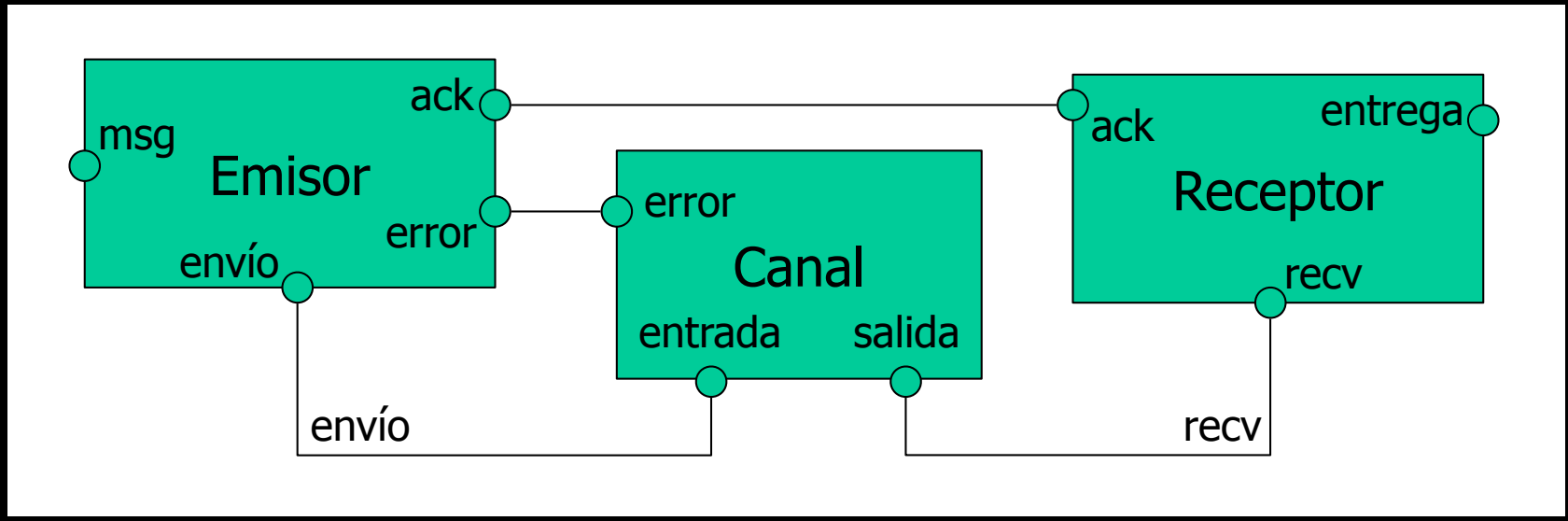
Diagrama arquitectónico - recursos compartidos



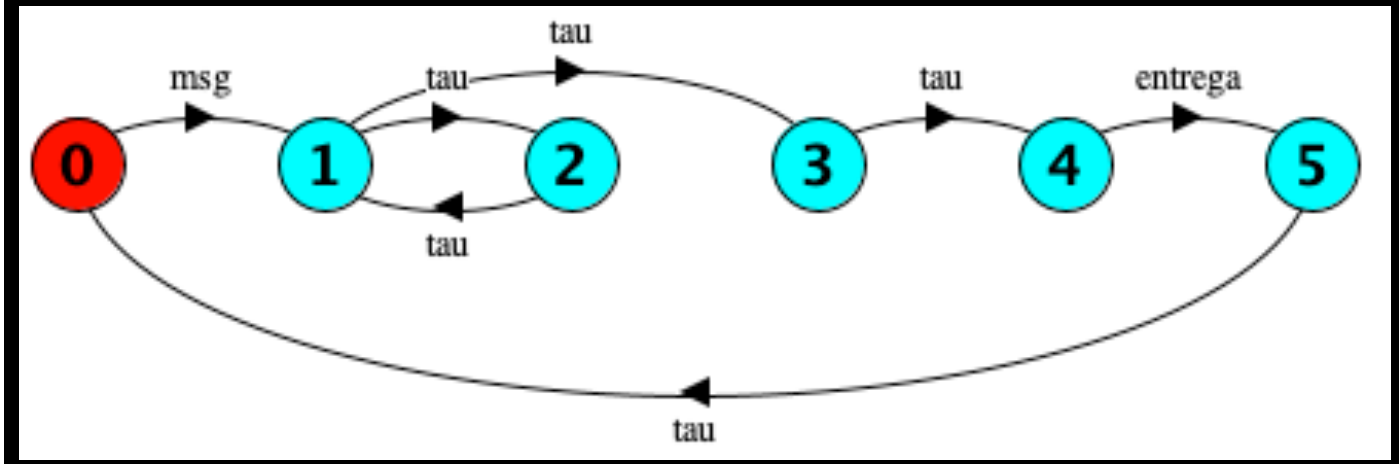

```
RESOURCE = (acquire->release->RESOURCE).
USER =     (printer.acquire->use
            ->printer.release->USER)\{use}.

||PRINTER_SHARE
  = (a:USER||b:USER||{a,b}::printer:RESOURCE).
```

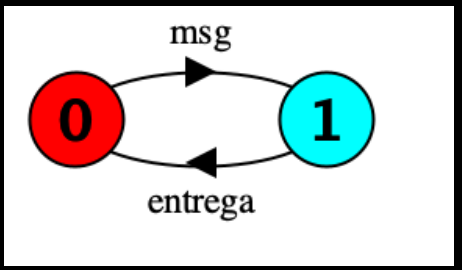
Verificación vía Bisimulación y Abstracción



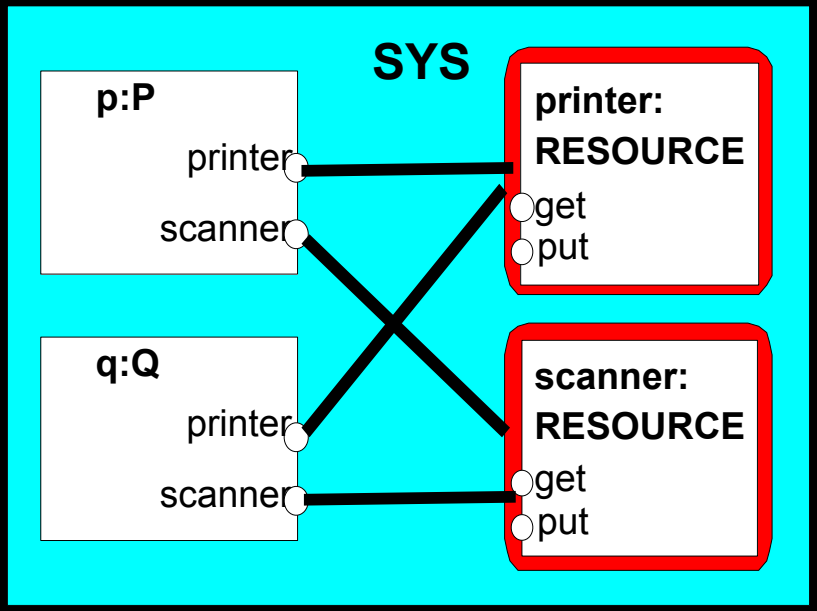

```
||ProtocoloAbstracto = Protocolo@{msg,entrega}.
```



≈



Deadlock



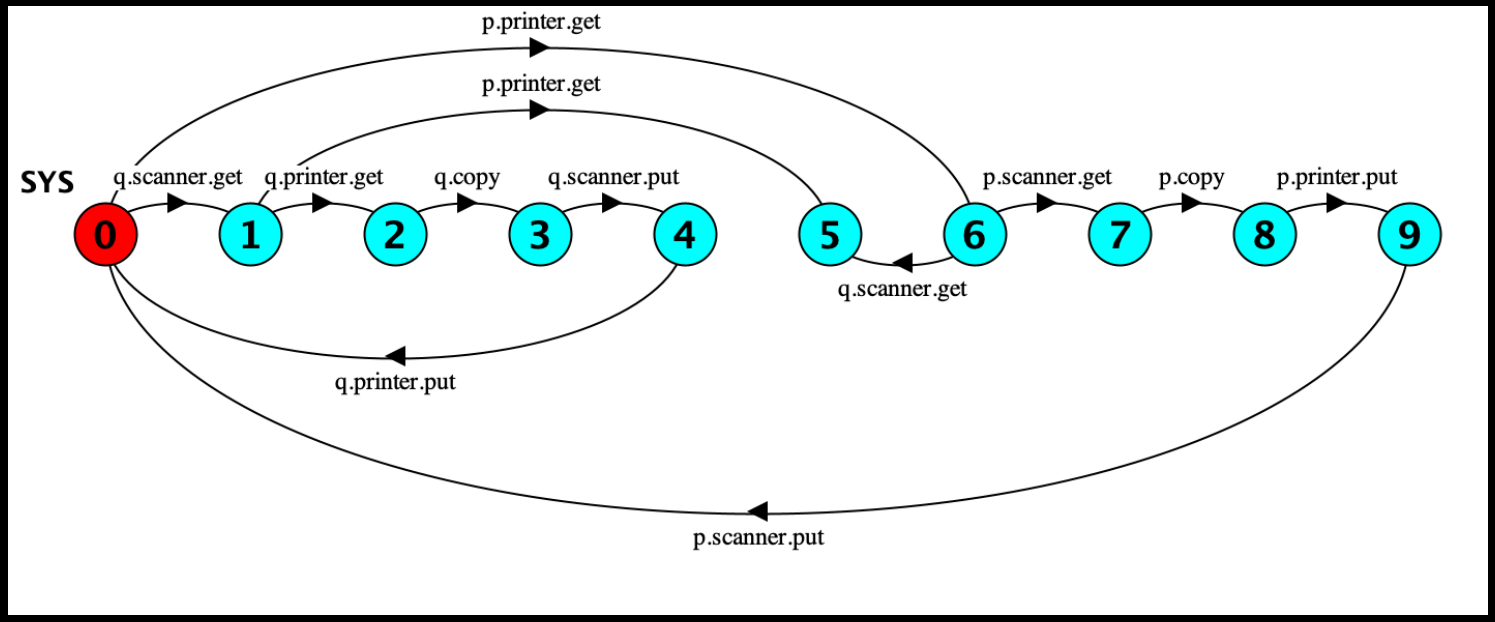

```
RESOURCE = (get->put->RESOURCE).

P = (printer.get->scanner.get
     ->copy
     ->printer.put->scanner.put->P).

Q = (scanner.get->printer.get
     ->copy
     ->scanner.put->printer.put->Q).

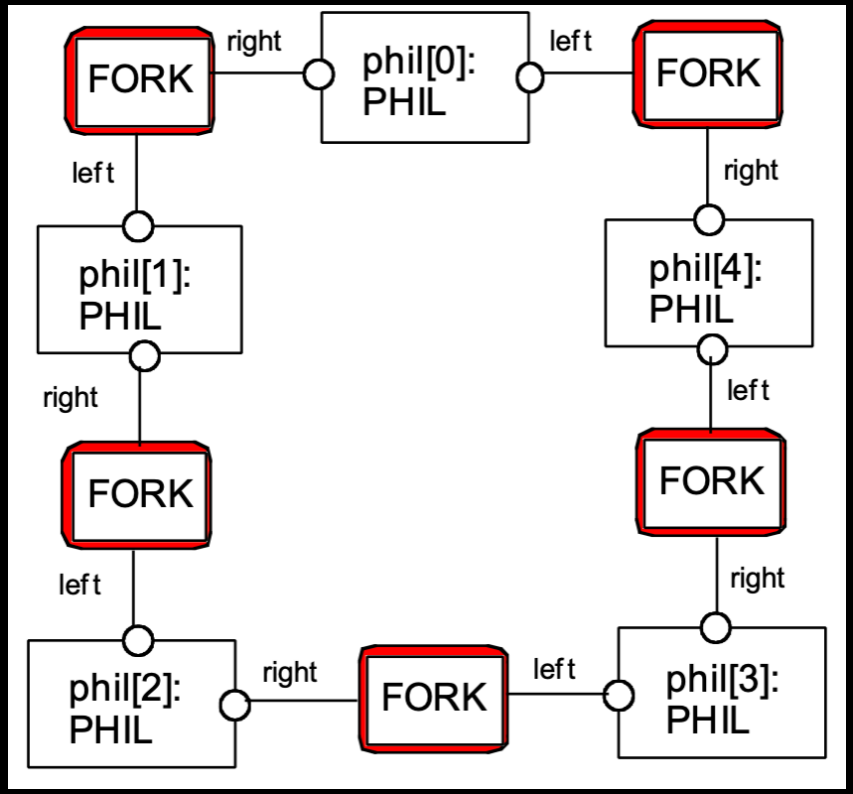
||SYS = (p:P||q:Q
   ||{p,q}::printer:RESOURCE
   ||{p,q}::scanner:RESOURCE
   ).
```

Deadlock Analysis



Una búsqueda breadth first search desde el estado inicial encuentra el camino más corto a deadlock.

Complejidad: $|\Delta|$



Los Filósofos

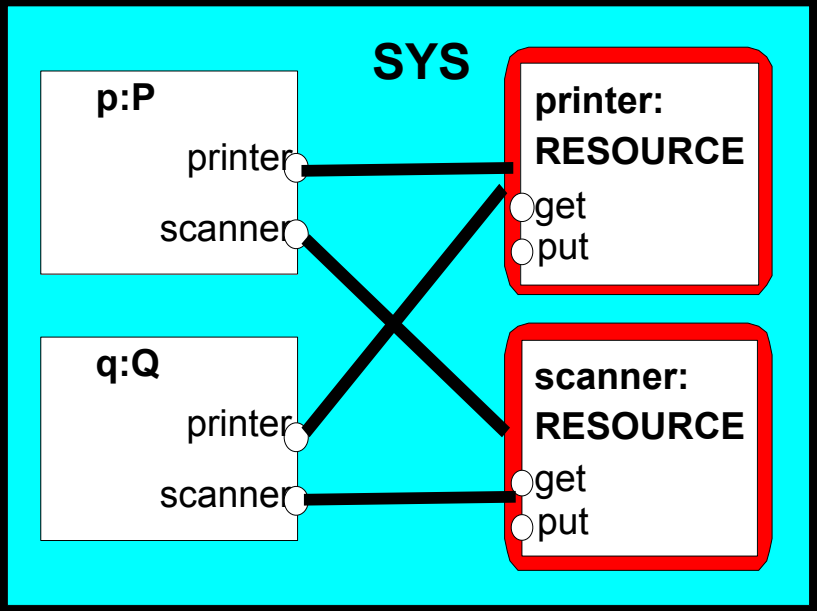
```
FORK = (get -> put -> FORK).
PHIL = (sitdown->right.get->left.get
        ->eat->right.put->left.put
        ->arise->PHIL).

||DINERS(N=5)= forall [i:0..N-1]
   (phil[i]:PHIL ||
   {phil[i].left,phil[((i-1)+N)%N].right}::FORK
   ).
```

Explosión de Estados

- La composición paralela de LTS produce una explosión de estados que en el peor caso es S^C con S el numero de estados de una componente y C el número de componentes.
- Existen algoritmos que evitan construir todo el espacio de estados. C.f. Partial order reduction

Deadlock




```
RESOURCE = (get->put->RESOURCE).

P = (printer.get->scanner.get
     ->copy
     ->printer.put->scanner.put->P).

Q = (scanner.get->printer.get
     ->copy
     ->scanner.put->printer.put->Q).

||SYS = (p:P||q:Q
   ||{p,q}::printer:RESOURCE
   ||{p,q}::scanner:RESOURCE
   ).
```

Solución de Deadlock

Timeouts

```
RESOURCE = (get->put->RESOURCE).

P          = (printer.get-> GETSCANNER),
GETSCANNER = (scanner.get->copy->printer.put
                                ->scanner.put->P
             |timeout -> printer.put->P
             )\{timeout}.
Q          = (scanner.get-> GETPRINTER),
GETPRINTER = (printer.get->copy->printer.put
                                ->scanner.put->Q
             |timeout -> scanner.put->Q
             )\{timeout}.

||SYS = (p:P||q:Q
   ||{p,q}::printer:RESOURCE
   ||{p,q}::scanner:RESOURCE
   ).
```

Solución de Deadlock

Preemption vía Watchdog

```
RESOURCE = (get->put->RESOURCE).

P          = (printer.get-> GETSCANNER),
GETSCANNER = (scanner.get->copy->printer.put
                              ->scanner.put->P
             |abort -> printer.put->P
             ).
Q          = (scanner.get-> GETPRINTER),
GETPRINTER = (printer.get->copy->printer.put
                              ->scanner.put->Q
             |abort -> scanner.put->Q
             ).

WATCHDOG = (abort -> WATCHDOG).

||SYS = (p:P||q:Q
   ||{p,q}::printer:RESOURCE
   ||{p,q}::scanner:RESOURCE
   ||{p,q}::WATCHDOG
   ).
```

Solución de Deadlock

Parametrización para romper simetría

```
PHIL(I=0)
  = (when (I%2==0) sitdown
        ->left.get->right.get
        ->eat
        ->left.put->right.put
        ->arise->PHIL
    |when (I%2==1) sitdown
       ->right.get->left.get
       ->eat
       ->left.put->right.put
       ->arise->PHIL
    ).


||DINERS(N=5)= forall [i:0..N-1]
    (phil[i]:PHIL ||
    {phil[i].left,phil[((i-1)+N)%N].right}::FORK
    ).
```

Puente 3.0 - Turnos

```
const True = 1
const False = 0
range B = False..True

BRIDGE = BRIDGE[0][0][0][0][True],
BRIDGE[nr:T][nb:T][wr:T][wb:T][bt:B] =
  (red[ID].request  -> BRIDGE[nr][nb][wr+1][wb][bt]
  |when (nb==0 && (wb==0||!bt))
     red[ID].enter  -> BRIDGE[nr+1][nb][wr-1][wb][bt]
  |red[ID].exit     -> BRIDGE[nr-1][nb][wr][wb][True]
  |blue[ID].request -> BRIDGE[nr][nb][wr][wb+1][bt]
  |when (nr==0 && (wr==0||bt))
     blue[ID].enter -> BRIDGE[nr][nb+1][wr][wb-1][bt]
  |blue[ID].exit    -> BRIDGE[nr][nb-1][wr][wb][False]
  ).
```

Algoritmo de Peterson para exclusión mutua

Lógica Temporal Lineal (LTL)

Lógica Modal



- Estudia la formalización del razonamiento que involucra aserciones con modalidades, tales como “necesariamente” o “posiblemente”

- Un enfoque tradicional es complementar los operadores “clásicos” con operadores **modales**:
  - [] : Necesidad (Box)
  - <> : Posibilidad (Diamond)

2.1 Linear Temporal Logic

Given a set of atomic propositions Π, a well-formed LTL formula is defined inductively using the standard Boolean operators, and the temporal operators X (next), [] (always), <> (eventually), U (until) and W (Awaits) as follows:

- each member of Π is a formula
- if P and Q are formulas, then so are ¬P, P∧Q, P∨Q, P → Q, P ↔ Q, X P, [] P, <> P, P U Q, P W Q.

An interpretation for an LTL formula is a infinite trace *h: Nat* -> 2^{Π} that maps to each position $i \in Nat$ the set of propositions that hold at that position. The notation (h,i) |= P is used to express that the LTL formula P is true at position i of the trace h. The semantics of the temporal logic operators is then defined as follows [19]:

- (h,i) |= X P *iff* (h, i+1) |=P
- (h,i) |= [] P *iff* (h, j) |=P *for all* j ≥ i
- (h,i) |= <> P *iff* (h, j) |=P *for some* j ≥ i
- (h,i) |= P *U* Q *iff* (h, j) |= Q *for some* j ≥ i
  and (h, k) |=P *for all k s.t.* i ≤ k < j
- (h,i) |= P *W* Q *iff* (h,i) |= P *U* Q *or* (h,i) |= []P

The Boolean operators have their usual semantics. A LTL formula P is said to be satisfied by a trace h, noted h |= P, if it is satisfied at the initial position, i.e. (h,0) |= P.